Carga de Tablas de Hechos al Data Warehouse

Sistema de Análisis de Ventas — Informe de carga de tablas de hechos

Proyecto:	Sistema de Análisis de Ventas (ETL con SQLite)
Entrega:	Carga de Tablas de Hechos al DW
Fecha:	Abril de 2026
Repositorio:	https://github.com/Lean-AZ/Proyecto_Ventas_ETL2
BD analítica:	data/ventas_analitica.db
Facts cargadas:	hechos_ventas · hechos_opiniones

1. Contexto del proceso

El Data Warehouse del proyecto almacena información analítica en **ventas_analitica.db** (SQLite), organizado en un esquema estrella. Las dimensiones (dim_producto, dim_cliente, dim_fecha) fueron cargadas en la etapa anterior. En esta entrega se documentan los pasos para cargar las **tablas de hechos**: hechos_ventas y hechos_opiniones.

Antes de cada carga se ejecuta un proceso de **limpieza (TRUNCATE lógico)** que elimina todas las filas de las tablas de hechos y resetea los contadores autoincrement. Esto garantiza una carga limpia y reproducible en cada ejecución del pipeline.

2. Arquitectura del proceso de carga

El script principal **cargar_facts_dw.py** orquesta el proceso en cuatro pasos secuenciales. Cada paso está aislado: si falla la carga de opiniones, los hechos de ventas ya están persistidos.

- **Paso 0 — Verificar schema**: se ejecuta sql/02_add_facts_opiniones.sql con CREATE TABLE IF NOT EXISTS para dim_fuente y hechos_opiniones. Las tablas ya existentes no se modifican.
- **Paso 1 — Limpieza de facts**: DELETE FROM hechos_ventas y DELETE FROM hechos_opiniones, seguido de DELETE FROM sqlite_sequence para resetear el autoincrement. Se registra la cantidad de filas eliminadas.
- **Paso 2 — Pipeline ETL**: ejecuta run_etl.py que extrae datos de CSV, BD relacional y API REST, los consolida en staging.db y los pasa a la BD analítica mediante etl/loader.py.
- **Paso 3 — Carga de hechos_ventas**: load_from_staging() lee staging_detalles JOIN staging_pedidos y genera filas para hechos_ventas con las FK correctas (producto_id, cliente_id, fecha_id en formato YYYYMMDD).
- **Paso 4 — Carga de hechos_opiniones**: cargar_hechos_opiniones() genera 800 registros sintéticos con calificaciones, sentimientos y comentarios en español, referenciando IDs reales de las dimensiones existentes.

Flujo de ejecución:

```
cargar_facts_dw.py
■■■ [Paso 0] asegurar_schema_opiniones() → dim_fuente, hechos_opiniones
■■■ [Paso 1] limpiar_todas_facts() → DELETE hechos_ventas + hechos_opiniones
■■■ [Paso 2] run_etl.py → CSV / BD / API → staging.db
■■■ [Paso 3] load_from_staging() → staging → hechos_ventas
■■■ [Paso 4] cargar_hechos_opiniones() → sintético → hechos_opiniones
```

3. Proceso de limpieza de tablas de hechos

La función **limpiar_todas_facts(conn)** en *etl/facts_loader.py* encapsula la lógica de limpieza. Antes de ejecutar cualquier INSERT, se eliminan todas las filas existentes y se restablece el contador de la clave primaria.

Lógica de limpiar_tabla_fact():

```
def limpiar_tabla_fact(conn, tabla):
    n = conn.execute(f'SELECT COUNT(*) FROM {tabla}').fetchone()[0]
    conn.execute(f'DELETE FROM {tabla}')   # elimina todas las filas
    conn.execute('DELETE FROM sqlite_sequence WHERE name=?', (tabla,))
    return n   # devuelve cantidad de filas eliminadas
```

Se usa **DELETE** en lugar de TRUNCATE porque SQLite no soporta TRUNCATE TABLE. El borrado de sqlite_sequence restablece el autoincrement al valor inicial (1), garantizando IDs consecutivos desde el principio en cada recarga.

Tablas limpiadas en cada ejecución:

Tabla de Hechos	Descripción
hechos_ventas	Líneas de detalle de ventas con FK a dim_producto, dim_cliente, dim_fecha
hechos_opiniones	Opiniones de clientes con FK a dim_producto, dim_cliente, dim_fecha, dim_fuente

Salida de log esperada después de la limpieza:

```
PASO 1: Limpieza de tablas de hechos
✓ hechos_ventas : 19,983 filas eliminadas
✓ hechos_opiniones : 800 filas eliminadas
Limpieza completada en 0.012 s
```

4. Tabla de hechos: hechos_ventas

Almacena cada línea de detalle de pedido del sistema de ventas. La granularidad es a nivel de producto-por-pedido: si un pedido tiene 3 productos distintos, se generan 3 filas en hechos_ventas.

Los datos se cargan mediante un JOIN entre staging_detalles y staging_pedidos, convirtiendo la fecha de texto ISO (YYYY-MM-DD) al entero YYYYMMDD que referencia dim_fecha.

Estructura de la tabla:

Columna	Descripción
id	PK surrogate autoincremental (reinicia en cada carga limpia)
id_pedido	Identificador del pedido; agrupa líneas del mismo pedido
producto_id	FK → dim_producto.id
cliente_id	FK → dim_cliente.id
fecha_id	FK → dim_fecha.id (formato YYYYMMDD del pedido)
cantidad	Unidades vendidas en esa línea de detalle
monto_total	Importe total de la línea (precio × cantidad)
creado_en	Timestamp de inserción (datetime SQLite)

SQL de carga (simplificado):

```
-- Paso previo: DELETE FROM hechos_ventas
INSERT INTO hechos_ventas
    (id_pedido, producto_id, cliente_id, fecha_id, cantidad, monto_total)
SELECT d.id_pedido, d.id_producto, p.id_cliente,
       CAST(REPLACE(p.fecha, '-', '') AS INTEGER),
       d.cantidad, d.monto_total
FROM staging_detalles d
JOIN staging_pedidos  p ON d.id_pedido = p.id_pedido;
```

4 filas)

producto_id	cliente_id	fecha_id
	348	20241115
	348	20241115
	4744	20250322
	4744	20250322
	1368	20241005
	1368	20241005
	1966	20240720
	1966	20240720
	4387	20240628
	580	20240814
	2806	20241015
	4446	20241122
	3277	20230929
	3277	20230929

Figura 1 — Muestra de filas en hechos_ventas (14 primeras)

la (JOIN con dimensiones)

	cliente	fecha	
	Melissa Kelley	2024-11-15	1
	Melissa Kelley	2024-11-15	6
	Henry Bryant	2025-03-22	7
	Henry Bryant	2025-03-22	2
	Calvin Martin	2024-10-05	1
	Calvin Martin	2024-10-05	8
	Shelly Atkinson	2024-07-20	7
	Shelly Atkinson	2024-07-20	1
	Lori Carr	2024-06-28	8
	Shelia Johnson	2024-08-14	8
	Daniel Andrews	2024-10-15	1
	Andrew Stephens	2024-11-22	6

Figura 2 — hechos_ventas desnormalizada: JOIN con dim_producto, dim_cliente, dim_fecha

ntas — top productos por ingresos

total_lineas	unidades_vendidas
38	234
38	231
41	235
38	220
35	227
40	218
37	222
43	236
46	236
37	244
41	220
34	216

Figura 3 — Top productos por ingresos totales (agrupado desde hechos_ventas)

5. Tabla de hechos: hechos_opiniones

Registra las opiniones de clientes sobre productos, capturadas desde múltiples fuentes (encuestas web, tienda online, redes sociales, app móvil). La granularidad es a nivel de opinión individual.

Los 800 registros cargados son sintéticos pero referencian IDs reales de las dimensiones existentes (dim_producto, dim_cliente, dim_fecha), garantizando la integridad referencial. El sentimiento se deriva automáticamente de la calificación: ≥4 → positivo, 3 → neutro, ≤2 → negativo.

Estructura de la tabla:

Columna	Descripción
id	PK surrogate autoincremental
producto_id	FK → dim_producto.id (producto evaluado)
cliente_id	FK → dim_cliente.id (nullable: 5% opiniones anónimas)
fecha_id	FK → dim_fecha.id (YYYYMMDD del comentario)
fuente_id	FK → dim_fuente.id (canal de captura)
calificacion	Puntuación 1–5 (entero con CHECK constraint)
sentimiento	positivo / neutro / negativo (CHECK constraint)
comentario	Texto libre en español
id_externo	Identificador del sistema de origen (EXT-NNNNN)
creado_en	Timestamp de inserción

s ›

	fecha_id	fuente_id	
	20241017	3	1
	20220324	2	5
	20230510	4	2
	20241130	3	2
	20210729	3	4
	20220626	4	1
	20210611	2	1
	20210404	3	5
	20230218	3	2
	20241004	3	3
	20250202	4	3
	20220326	2	2
	20221009	1	3
	20251130	4	4

Figura 4 — Muestra de filas en hechos_opiniones (14 primeras)

zada (JOIN con dimensiones)

	fuente	calificacion	
	Redes Sociales	1	n
	Tienda Online	5	p
	App Móvil	2	n
	Redes Sociales	2	n
	Redes Sociales	4	p
	App Móvil	1	n
	Tienda Online	1	n
	Redes Sociales	5	p
	Redes Sociales	2	n
	Redes Sociales	3	n
	App Móvil	3	n
	Tienda Online	2	n

Figura 5 — hechos_opiniones desnormalizada: JOIN con dimensiones

iniones — distribución por sentimiento

	total_opiniones	calif_promedio	
	321	4.5	
	318	1.49	
	161	3.0	

Figura 6 — Distribución de sentimientos con calificación promedio por grupo

6. Dimensión adicional: dim_fuente

Para soportar hechos_opiniones se creó la dimensión **dim_fuente**, que cataloga los canales desde donde se capturan las opiniones de los clientes.

Estructura de dim_fuente:

Columna	Descripción
id	PK autoincremental
nombre	Nombre del canal (UNIQUE): Encuesta Web, Tienda Online, etc.
descripcion	Descripción del canal de captura
creado_en	Timestamp de inserción

Fuentes registradas:

Nombre	Descripción
Encuesta Web	Formulario de satisfacción post-compra en el sitio web
Tienda Online	Reseñas publicadas en la plataforma de e-commerce
Redes Sociales	Menciones y comentarios en Twitter/Instagram
App Móvil	Calificaciones enviadas desde la aplicación móvil

ouse — dim_fuente (catálogo de fuentes de o

	nombre	descr
	Encuesta Web	Formulario de s
	Tienda Online	Reseñas public
	Redes Sociales	Menciones y co
	App Móvil	Calificaciones e

Figura 7 — dim_fuente en ventas_analitica.db

7. Archivos involucrados en el proceso

Archivo	Rol en el proceso
`cargar_facts_dw.py`	Script principal. Orquesta limpieza + carga de ambas facts.
`etl/facts_loader.py`	Funciones de limpieza (limpiar_todas_facts) y carga (cargar_hechos_opiniones).
`etl/loader.py`	Función load_from_staging: carga hechos_ventas desde staging.db.
`sql/02_add_facts_opiniones.sql`	Schema SQL: CREATE TABLE dim_fuente y hechos_opiniones con índices.
`sql/01_create_schema_ventas.sql`	Schema base: dim_producto, dim_cliente, dim_fecha, hechos_ventas.
`run_etl.py`	Pipeline ETL: extracción CSV/BD/API → staging → carga analítica.
`etl/staging.py`	Gestión de staging.db: tablas staging_productos, clientes, pedidos, detalles.
`scripts/capturas_facts_dw.py`	Genera PNG de las tablas de hechos para esta documentación.

8. Cómo reproducir el proceso

- Clonar el repositorio: `git clone https://github.com/Lean-AZ/Proyecto_Ventas_ETL2`
- Instalar dependencias: `pip install reportlab matplotlib python-docx`
- Ejecutar carga completa: `python cargar_facts_dw.py`
- Para omitir la extracción ETL (reutilizar staging): `python cargar_facts_dw.py --skip-etl`
- Regenerar este PDF: `python scripts/generar_pdf_carga_facts_dw.py`